02 Final Result Report

Final outputs, architecture summary, setup guide, API evidence, limitations, and improvements.

Repository	https://github.com/aalanani2000/al-rouf-ai-integration-assessment
Default branch	main
Latest commit	653ec552871083dca14498d120d6e6c156ed4f31
Prepared date	2026-06-24
Assessment	AI Integration Engineer task pack

Final Outcome

The assessment implementation is complete across all three requested tasks. The repository contains runnable source code, Docker packaging, tests where required, exported workflow artifacts, setup documentation, screenshots, and final reports.

Task	Implementation	Validation	Status
Task 1	n8n RFQ webhook, OpenAI extraction, mock CRM, bilingual reply, alert log.	Successful n8n executions, persisted JSON records, screenshot evidence.	Complete; attachment bytes scoped as future enhancement.
Task 2	FastAPI quotation API with catalog, pricing tiers, SQLite, Docker, OpenAPI.	Automated pytest suite and OpenAPI screenshot evidence.	Complete.
Task 3	FastAPI bilingual RAG service over English/Arabic documents with citations and refusal.	Unit tests, Arabic question evidence, error-handling screenshot.	Complete.

Task 1 Final Result

- Inbound RFQ data enters through an n8n webhook.
- OpenAI extracts urgency, key requirements, and summary from the RFQ message.
- The workflow creates a mock CRM record, bilingual reply draft, and internal alert.

Task 2 Final Result

- FastAPI exposes health, catalog, quote creation, and quote retrieval endpoints.
- Bulk discount tiers are applied by total quote quantity.
- SQLite stores generated quotes; OpenAPI documentation is available at `/docs`.

Task 3 Final Result

- The service indexes 4 English/Arabic source documents.

- Responses cite source documents and include latency and estimated cost metadata.
- Out-of-scope questions are refused when retrieval confidence is insufficient.

Architecture Explanation

- Task 1 uses n8n because the workflow can be exported, versioned, reviewed, and rerun locally.
- Task 2 separates schemas, catalog data, pricing logic, database access, and route handlers.
- Task 3 keeps the RAG system lightweight because the assessment document set is small.

Run And Test Commands

Area	Command	Purpose
Task 1	docker-compose up -d	Start n8n from repository root.
Task 2	cd task2_quotation_microservice && docker-compose up --build	Run quotation API on port 8000.
Task 2 tests	cd task2_quotation_microservice && PYTHONPATH=. pytest tests/ -v	Run quotation tests.
Task 3	cd task3_bilingual_rag && docker-compose up --build	Run RAG API on port 8001.
Task 3 tests	cd task3_bilingual_rag && PYTHONPATH=. pytest tests/ -v	Run RAG tests.

Repository Guidance

Use this structure map to navigate the solution bundle and GitHub repository:

```
C:.
| .env
| .env.example
| .gitignore
| docker-compose.yml
| README.md
|
+---data
| alerts.json
| mock_crm.json
|
+---docs
| +---architecture
| +---evidence
| | \---task1_rfq_crm_automation
| | execution_notes.md
| |
| \---screenshots
| +---Task 1
| | javascript code 1 error .png
| | n8n canva.png
| | open ai node.png
| | proof of the volumes being correctly attached.png
| | repository structure.png
| | set field output.png
| | set field.png
| | Webhook node.png
| | workflow progress .png
| | workflow progress 1 .png
| | workflow succes.png
| | workflow.png
| | workflow2.png
| |
| +---Task 2
| | Open API.png
| |
| \---Task 3
| a problem when communicating with model.png
| testing the RAG on arabic question.pdf
|
+---reports
| +---01_execution_evidence_report
| \---02_final_result_report
+---task1_rfq_crm_automation
| \---n8n_workflows
| rfq-crm-automation.json
|
+---task2_quotation_microservice
| | .gitignore
| | docker-compose.yml
| | Dockerfile
| | README.md
| | requirements.txt
| |
| +---app
| | catalog.py
| | database.py
| | main.py
| | quoting.py
| | schemas.py
| | __init__.py
| |
| +---data
| | quotes.db
| |
| \---tests
| test_quotation.py
| __init__.py
|
\---task3_bilingual_rag
| .env
| .env.example
| .gitignore
| docker-compose.yml
| Dockerfile
| README.md
| requirements.txt
|
+---app
| embeddings.py
| llm.py
| loader.py
| main.py
| schemas.py
| __init__.py
```

```
|
+---data
| embeddings_cache.json
|
+---documents
| doc1_product_spec_high_bay_en.md
| doc2_warranty_policy_ar.md
| doc3_shipping_terms_en.md
| doc4_product_spec_street_light_ar.md
|
\---tests
test_rag.py
__init__.py
```

Error Handling, Maintainability, And Security

- Task 1 persists n8n state and mock CRM data with Docker volumes and bind mounts.
- Task 2 validates input with Pydantic and returns clear HTTP errors.
- Task 3 refuses unsupported questions through retrieval thresholding.
- Secrets are stored in `.env` files or runtime environment variables, not hardcoded.

Future Enhancements

- Task 1: ingest RFQs from real email inboxes, WhatsApp Business, website forms, or CRM webhooks.
- Task 1: archive actual attachment files and add duplicate detection.
- Task 2: add a quotation UI, PDF quote generation, authentication, configurable pricing, taxes, shipping, and ERP/CRM integration.
- Task 3: add a polished bilingual chat UI, document upload, richer citations, streaming responses, access control, and a vector database if the corpus grows.

AI Assistance Disclosure

AI assistance was used for brainstorming architecture choices, debugging explanations, starter snippets, and documentation/report polish. The candidate remained responsible for all local execution, implementation, testing, configuration, screenshots, trade-off selection, and final submission packaging.